CUED - Engineering Tripos Part IIB 2025-2026 **Module Coursework**

Module	4C11	Title of report	4C11 CW2 Data-driven and learning based methods
Date submitted: 12/03/26		Assessment for this module is ☐ 100% / ☐ 25% coursework of which this assignment forms 33 %	

UNDERGRADUATE and POST GRADUATE STUDENTS		
Candidate number:	3233J	☒ Undergraduate ☐ Post graduate

Feedback to the student ☐ **See also comments in the text**		Very good	**Good**	Needs improvmt
CONTENT	**Completeness, quantity of content:** Has the report covered all aspects of the lab? Has the analysis been carried out thoroughly?			
	Correctness, quality of content Is the data correct? Is the analysis of the data correct? Are the conclusions correct?			
	Depth of understanding, quality of discussion Does the report show a good technical understanding? Have all the relevant conclusions been drawn?			
	Comments:			
PRESENTATION	**Attention to detail, typesetting and typographical errors** Is the report free of typographical errors? Are the figures/tables/references presented professionally?			
	Comments:			

Marker: Date:

4C11 CW2: Data-driven and learning based methods

Contents

1 Solving PDEs: Plate with a hole **2**
- 1.1 Problem Definition . . . 2
  - 1.1.1 PINN Architecture . . . 2
  - 1.1.2 Derivation of the Governing Equation . . . 3
  - 1.1.3 Boundary Conditions . . . 4
- 1.2 Loss Function . . . 4
- 1.3 PINN Performance Results . . . 5
- 1.4 Augmenting with FEA Data. . . . 6

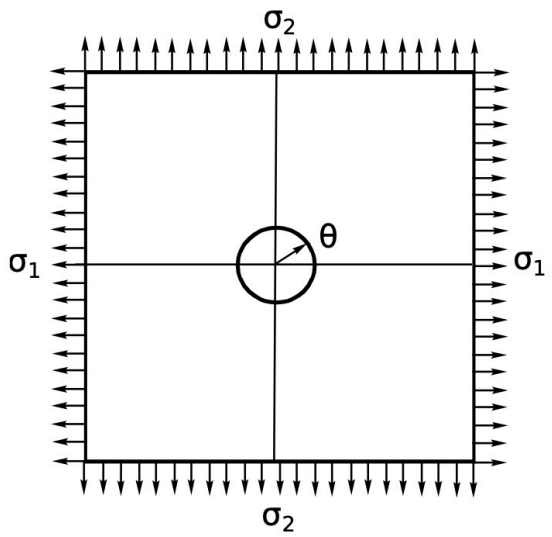
2 Learning PDEs: 2D Darcy flow **7**
- 2.1 Problem Definition . . . 7
- 2.2 Interpreting Performance . . . 8
- 2.3 Convolutional Neural Network (CNN) . . . 9
  - 2.3.1 Architecture Choice . . . 9
  - 2.3.2 Optimiser and Learning Rate . . . 9
  - 2.3.3 Results . . . 9
- 2.4 Fourier Neural Operator (FNO) . . . 11
  - 2.4.1 Architecture choice . . . 11
  - 2.4.2 Optimiser and Learning Rate . . . 12
  - 2.4.3 Results . . . 12

1 Solving PDEs: Plate with a hole

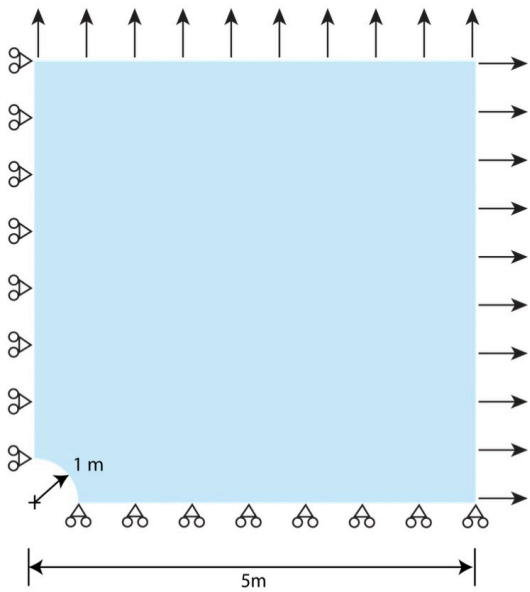
1.1 Problem Definition

The objective of this project is to write a Physics-informed Neural Network (PINN) solver to solve the 2D linear elastic boundary value problem depicted in Figure 1a (under plane stress assumptions). By symmetry, it is sufficient to simulate only one quadrant of the problem with uniform traction applied on two edges and sliding essential boundary conditions applied on the other two edges, as shown in Figure 1b. We use the provided MATLAB script `Plate_hole.m` to generate the collocation points (Figure 3a) alongside their respective stresses. We are also provided with a ground truth solution from the Finite Element Analysis (Figure 3b).

Young's modulus, Poisson's ratio and plate thickness are $E = 10\,\mathrm{N/m^2}$, $\nu = 0.3$, and $t = 1\,\mathrm{m}$, respectively. We use $\sigma_1 = 0.1\,\mathrm{N/m^2}$ and $\sigma_2 = 0\,\mathrm{N/m^2}$.



(a) Plate with a circular hole.



(b) Numerical model with dimensions.

1.1.1 PINN Architecture

The PINN template `PINN_skeleton.py` we are provided with implements an advanced PINN architecture to prevent optimisation over second-order derivatives of neural network outputs, which is known to be challenging. This is done by implementing two separate fully connected neural networks (FCNNs):

1. **Displacement Network:** The first network maps the spatial coordinates (x, y) to displacement (u, v).
2. **Stress Network:** The second network separately maps the spatial coordinates (x, y) to the stress components $(\sigma_{xx}, \sigma_{yy}, \tau_{xy})$.

To ensure consistency between the two independent networks, we first differentiate the output of the displacement network to predict the strains $(\epsilon_{xx}, \epsilon_{yy}, \gamma_{xy})$. We then use Hooke's law to calculate an 'augmented stress' field, and constrain the output of the stress network to match this using the loss function.

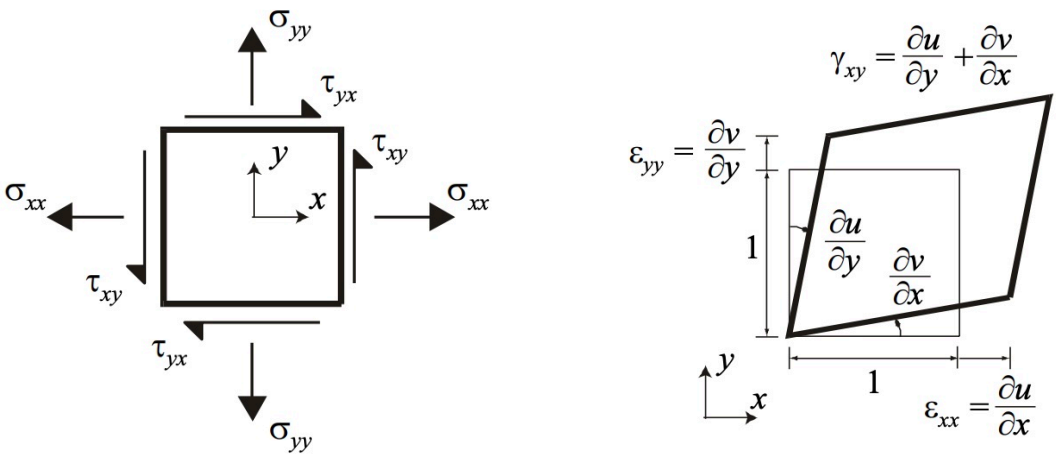
1.1.2 Derivation of the Governing Equation

We first consider an infinitesimal 2D element with dimensions $dx \times dy$ and uniform thickness t (as in the Structures Databook Figure 2a). For equilibrium we require that the forces in the x-direction cancel ($\sum F_x = 0$). Using a first-order Taylor series expansion to represent the change in stress across dx and dy, and cancelling the base stresses in one step, we derive:

$$\sum F_x = \left(\frac{\partial \sigma_{xx}}{\partial x} dx\right) dy \cdot t + \left(\frac{\partial \tau_{xy}}{\partial y} dy\right) dx \cdot t = 0$$

Simplifying by dividing by the elemental volume, and by similar logic for the y-direction, we arrive at the following two PDEs which must be satisfied by the stress network for every collocation point:

$$\frac{\partial \sigma_{xx}}{\partial x} + \frac{\partial \tau_{xy}}{\partial y} = 0 \qquad \frac{\partial \tau_{xy}}{\partial x} + \frac{\partial \sigma_{yy}}{\partial y} = 0 \tag{1}$$



(a) Infinitesimal material block.

(b) Small strain relations.

As Figure 2b (also taken from the Structures Databook) shows, we can use the small strain relations to map from displacements u and v to strains. This is used to determine the strain from the displacement network:

$$\epsilon_{xx} = \frac{\partial u}{\partial x}, \quad \epsilon_{yy} = \frac{\partial v}{\partial y}, \quad \gamma_{xy} = \frac{\partial u}{\partial y} + \frac{\partial v}{\partial x} \tag{2}$$

And to calculate the 'augmented stresses' from the strain prediction, assuming a linear elastic, isotropic material under plane stress conditions, we apply Hooke's Law:

$$\sigma_{xx} = \frac{E}{1-\nu^2}(\epsilon_{xx} + \nu\epsilon_{yy}) \tag{3}$$

$$\sigma_{yy} = \frac{E}{1-\nu^2}(\epsilon_{yy} + \nu\epsilon_{xx}) \tag{4}$$

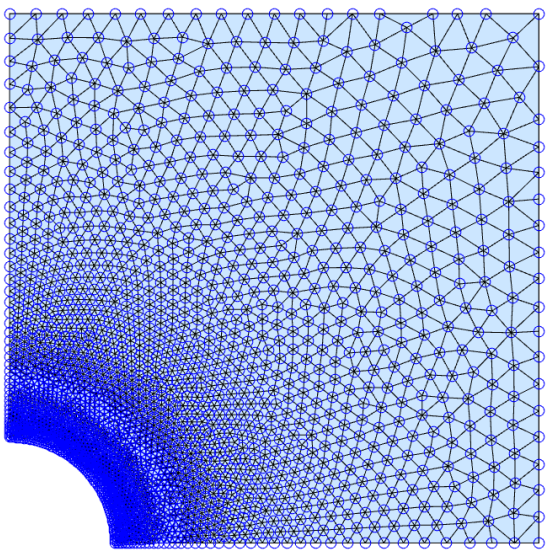
$$\tau_{xy} = G\gamma_{xy} = \frac{E}{2(1+\nu)}\gamma_{xy} \tag{5}$$

where E is Young's modulus, ν is Poisson's ratio, and G is the shear modulus.

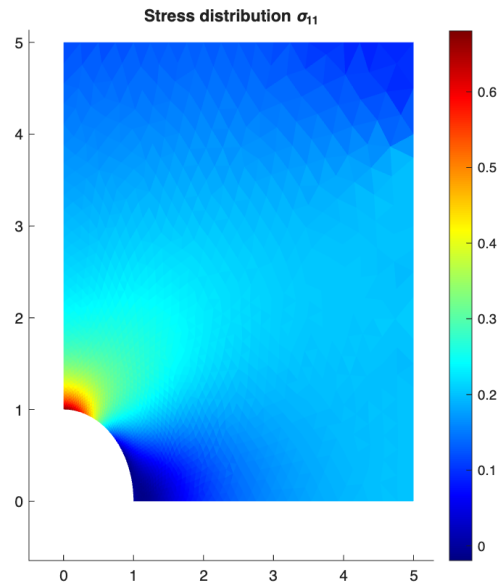
1.1.3 Boundary Conditions

Based on the symmetry of the problem and the applied loads, the domain is a $5\,\mathrm{m} \times 5\,\mathrm{m}$ quadrant with a $1\,\mathrm{m}$ radius hole at the origin. The boundary conditions are given by:

1. **Left Symmetry:** $u = 0, \quad \tau_{xy} = 0 \quad \text{on } x = 0,\ y \in [1, 5]$
2. **Bottom Symmetry:** $v = 0, \quad \tau_{xy} = 0 \quad \text{on } y = 0,\ x \in [1, 5]$
3. **Right:** $\sigma_{xx} = \sigma_1 = 0.1\,\mathrm{N/m^2}, \quad \tau_{xy} = 0 \quad \text{on } x = 5,\ y \in [0, 5]$
4. **Top:** $\sigma_{yy} = \sigma_2 = 0\,\mathrm{N/m^2}, \quad \tau_{xy} = 0 \quad \text{on } y = 5,\ x \in [0, 5]$
5. **Hole Boundary:** $\sigma_{xx}x + \tau_{xy}y = 0, \quad \tau_{xy}x + \sigma_{yy}y = 0 \quad \text{on } x^2 + y^2 = 1,\ x, y \geq 0,$ this is derived from the Cauchy traction equations.



(a) Map of collocation points.



(b) Ground truth stress distribution from Finite Element Analysis (FEA).

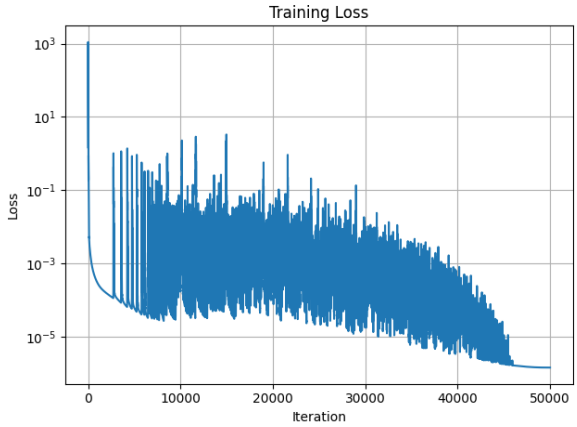
1.2 Loss Function

The loss function we use to train our PINN is a sum of three terms, for each term we use the Mean Squared Error (MSE) since it penalises large errors more significantly:

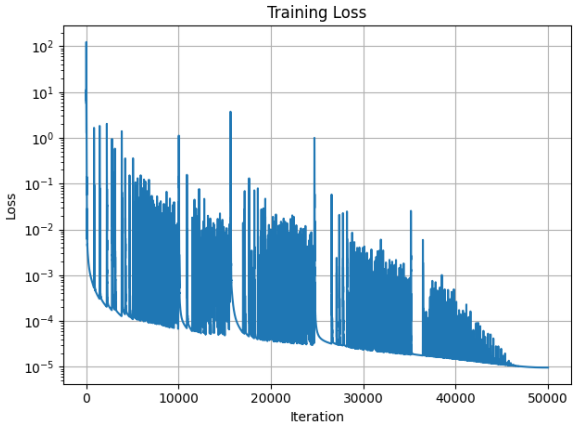
$$\mathcal{L} = \mathcal{L}_{\text{Residual}} + \mathcal{L}_{\text{Boundary}} + \mathcal{L}_{\text{Consistency}} \tag{6}$$

1. **Residual Loss**($\mathcal{L}_{\text{Residual}}$): Enforces force equilibrium through the two PDEs (Equation 1) derived earlier. We use the MSE over all the collocation points.
2. **Boundary Loss**($\mathcal{L}_{\text{Boundary}}$): Enforces the required physical constraints across all five boundaries discussed in subsubsection 1.1.3. It is the MSE over all collocation points on each respective boundary.
3. **Consistency Loss**($\mathcal{L}_{\text{Consistency}}$): The MSE between the 'augmented stresses' calculated from the displacement network and the output of the stress network, for every collocation point, to enforce consistency between them.

1.3 PINN Performance Results



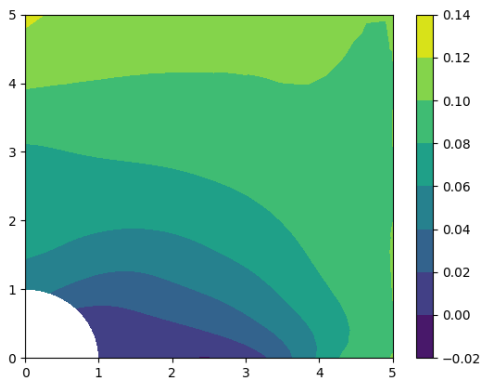
(a) Training loss against training iterations, learning rate of 0.005.



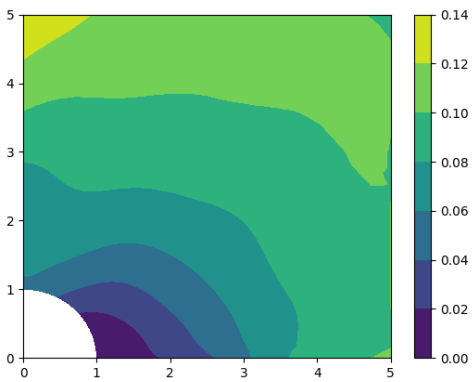
(b) Training loss against training iterations, learning rate of 0.001, reweighted loss.

For broad hyperparameter tuning we experience a prohibitively large training time, on the order of 3 hours. This rules out use of the `Optuna` Bayesian hyperparameter tuning framework discussed in the previous coursework. Instead we choose to test a few different setups with a fixed training budget of $50,000$ iterations, and compare their loss curves. We use the `Adam` optimiser and `CosineAnnealingLR` scheduler in all cases.

In the first case, with default values and a learning rate of 0.005, we obtain the loss shown in Figure 4a. This produces an output stress distribution of Figure 5a which broadly resembles the ground truth from Finite Element Analysis (FEA) in Figure 3b. However, it excludes the high stress region above the hole, and an incorrectly predicts a high stress at the upper plate boundary.



(a) Output stress distribution, learning rate of 0.005.



(b) Output stress distribution, learning rate of 0.001, reweighted loss.

The loss curve shows that training progresses, saturating at the budget of $50,000$ epochs. However, the gradients are clearly unstable, resulting in sharp spikes in the loss. Another issue is determined by inspecting the components of the loss at the first step of training:

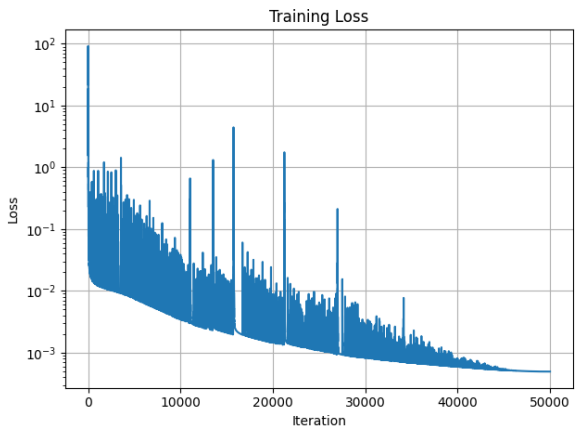
$$\mathcal{L}_{\text{Residual}} = 0.0120 \qquad \mathcal{L}_{\text{Boundary}} = 0.4747 \qquad \mathcal{L}_{\text{Consistency}} = 1.5554 \tag{7}$$

The magnitudes of the different components differ significantly, which affects their relative importance as perceived by the optimiser. In this case, the PDE loss ($\mathcal{L}_{\text{Residual}}$) is small enough such that its gradients will effectively be ignored, meaning that the internal physics the model produces are unlikely to be accurate. Furthermore, $\mathcal{L}_{\text{Consistency}}$, which constrains the displacement and stress networks to agree with each other, is relatively large. This creates a risk that the networks both learn to trivially output a stress of 0 over the domain, solving the consistency problem while ignoring the physics.

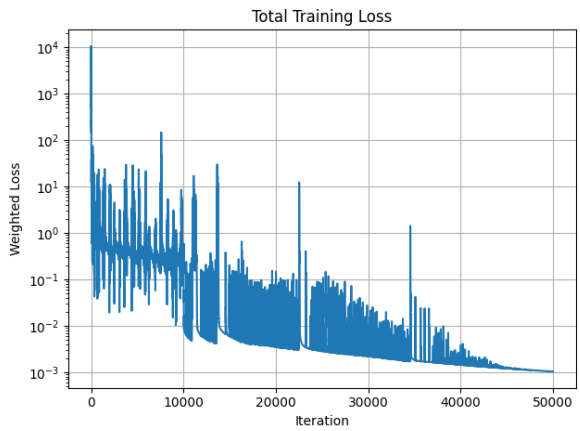
To mitigate this, we have retrained with a lower learning rate (to stabilise the loss) and a re-weighted loss function, which scales up the PDE loss ($\mathcal{L}_{\text{Residual}}$) by a factor of 50, and the boundary loss ($\mathcal{L}_{\text{Boundary}}$) by a factor of 10. The resulting stress distribution and loss curve are shown in Figure 4b and Figure 5b. The training looks marginally more stable, and the output stress distribution is a closer match with the ground truth.

However, there is significant room for improvement, one suggestion would be to use gradient clipping (`nn.utils.clip_grad_norm`) to prevent gradient explosion, and possibly to dynamically re-weight the loss terms as training progresses.

1.4 Augmenting with FEA Data.



(a) Training loss against training iterations with FEA data exposure, learning rate of 0.001.

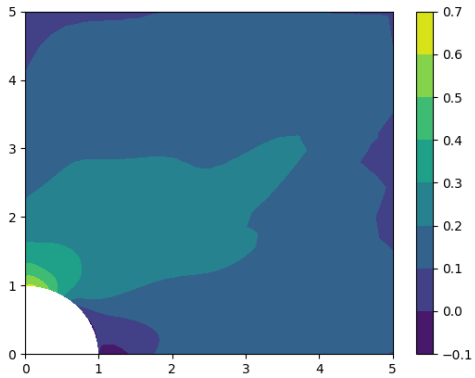


(b) Training loss against training iterations with FEA data exposure, learning rate of 0.002 and gradient clipping.

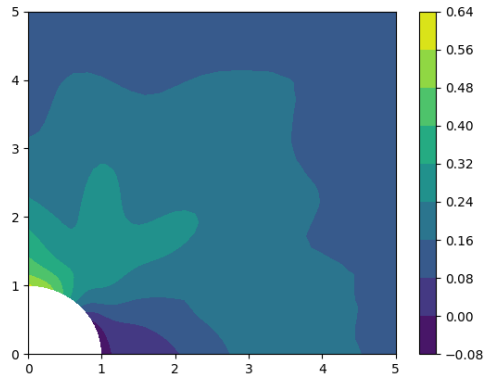
To improve the performance of the PINN, we are provided with measurement data of the displacement field $u(x)$ at 50 points in the domain, calculated using Finite Element Analysis (FEA). We incorporate this information into the training process by adding a new MSE 'data loss' term $\mathcal{L}_{\text{Data}}$, such that:

$$\mathcal{L} = \mathcal{L}_{\text{Residual}} + \mathcal{L}_{\text{Boundary}} + \mathcal{L}_{\text{Consistency}} + \mathcal{L}_{\text{Data}} \tag{8}$$

We find this yields a significant performance improvement. Figure 6a shows the initial PINN loss curve, while Figure 6b shows the loss curve after some hyperparameter tuning. The tuning included increasing the neural network sizes and the learning rate ($0.001 \rightarrow 0.002$), and also weighting the data loss $\mathcal{L}_{\text{Data}}$ more heavily. Note this again means that the final values of the losses are not directly comparable between pre and post-tuned models. We also introduce the gradient clipping in an effort to stabilise training.



(a) Output stress distribution after FEA data training, learning rate of 0.001.



(b) Output stress distribution after FEA data training, LR of 0.002 and gradient clipping.

The resulting stress distributions are shown in Figure 7a and Figure 7b. After the addition of the FEA data it is clear that the stress distributions both more closely resemble the ground truth (Figure 3b). It is also clear that the model performs better after hyperparameter tuning, with an even more refined output stress distribution.

2 Learning PDEs: 2D Darcy flow

2.1 Problem Definition

Let us consider learning the solution operator of a 2D Darcy flow problem. The Darcy equations form a model used to describe the flow of fluids through porous substances, useful for optimal extraction of oil, for example. We consider the following 2-D Darcy flow equation on the unit box with Dirichlet boundary conditions:

$$-\nabla \cdot (a(\mathbf{x})\nabla u(\mathbf{x})) = f(\mathbf{x}) \quad \mathbf{x} \in (0,1)^2, \tag{9}$$

$$u(\mathbf{x}) = 0 \quad \mathbf{x} \in \partial(0,1)^2. \tag{10}$$

where $a(\mathbf{x})$ is the diffusion coefficient generated from a random Fourier field while $f(\mathbf{x})$ is the forcing function which we keep as a constant throughout the domain. Our aim is to learn the operator mapping the diffusion coefficient to the solution defined by:

$$\Psi^\dagger : a \mapsto u,$$

We are provided a training and test dataset, which are generated from the MATLAB Finite Element Analysis (FEA) script "`Darcy_2D.m`". Note here we are not developing a Physics-informed network, instead we are using supervised learning from the FEA data.

2.2 Interpreting Performance

When evaluating the performance of our models, we would normally directly compare the ground truth solution from FEA with the model output (Figure 8). However, it is clear that the solutions $u(\mathbf{x})$ appear very similar irrespective of the input $a(\mathbf{x})$. Consequently, it is difficult to tell visually if our model approximation $\hat{u}(\mathbf{x})$ is performing well, or just matching the 'average' solution.

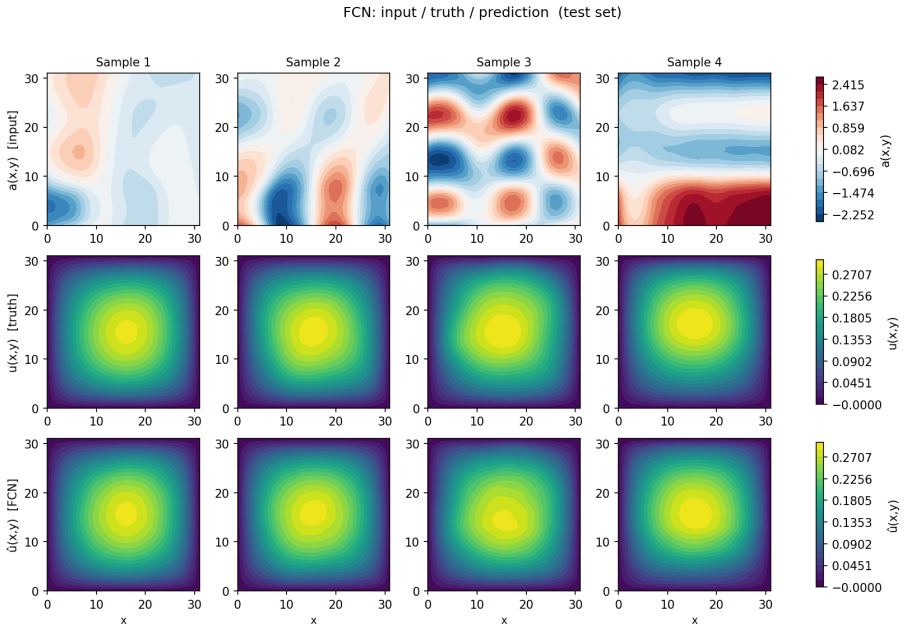


Figure 8: Plot of the input diffusion coefficient map $a(\mathbf{x})$, the FEA ground truth flow solution $u(\mathbf{x})$ and the approximate solution derived with an FCN architecture $\hat{u}(\mathbf{x})$.

A fix is to plot the solution and its approximation after they have been passed through our `UnitGaussianNormalizer` (Figure 9). This effectively removes the 'average shape' from every sample, and since our model is trained in the normalised space we can more directly evaluate its performance:

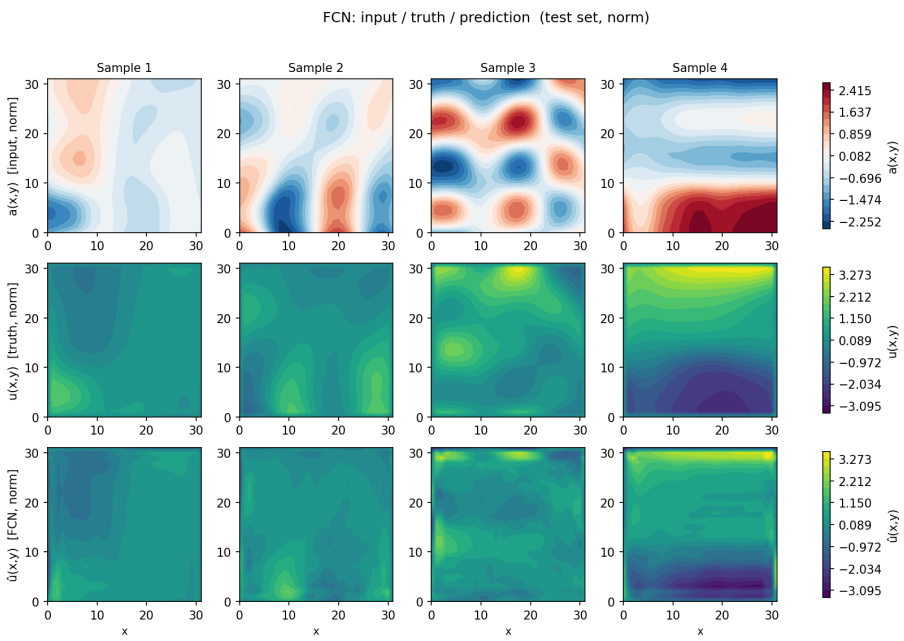


Figure 9: Plot of the input $a(\mathbf{x})$, the normalised FEA ground truth solution $u(\mathbf{x})$ and the normalised approximation derived with an FCN architecture $\hat{u}(\mathbf{x})$.

2.3 Convolutional Neural Network (CNN)

2.3.1 Architecture Choice

To learn the solution operator $\Psi^\dagger : a \mapsto u$, we implement both a baseline Fully Convolutional Network (FCN) and a U-Net. Both are convolutional architectures, exploiting the translation invariance of local features to reduce parameter count compared to a FCNN:

- **Fully Convolutional Network (FCN):** Uses consecutive convolutions that preserve the 32×32 input spatial dimensions. It is the simplest architecture to implement, hence is used as a baseline, but struggles to observe global features.
- **U-Net:** Based on the previous coursework, but in 2D with no classification head. The encoder-decoder structure expands the receptive field to capture global features, while skip connections retain high-resolution local details.

For the FCN the training time is small enough that we can use the `Optuna` Bayesian optimisation framework to run a number of trials with different hyperparameters and tune for the lowest test loss. We tune the following hyperparameters:

- **Channel Width:** The number of feature maps within the each convolutional layer.
- **Network Depth:** The total number of stacked convolutional layers.
- **Kernel Size:** The spatial dimensions of the convolutional filters (restricted to odd integers to maintain symmetric padding).
- **Learning Rate**

The tuned FCN has a parameter count of $7,300$, and we use this as a heuristic to fix the size of the U-Net architecture, such that we balance the risk of overfitting against the need for sufficient model complexity to avoid underfitting.

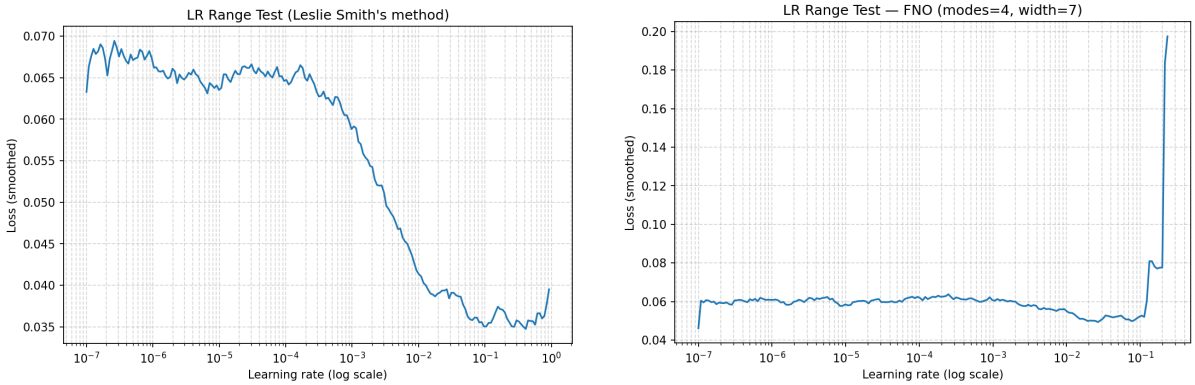
2.3.2 Optimiser and Learning Rate

We train with the `Adam` optimiser, which uses adaptive learning rates and momentum to converge efficiently (even if the loss is much steeper in certain directions) without getting stuck in local minima.

To avoid an expensive grid search for the optimum U-Net learning rate, we instead implement a learning rate finder proposed by Leslie N. Smith. This works by exponentially increasing the learning rate batch-by-batch over a set number of iterations, and plotting the loss (Figure 10a). At low learning rate the loss remains high. Then we have a region, between learning rates of 0.0002 and 0.01 where the loss decreases significantly, before it stagnates again at higher rates. This gives us sensible bounds on the learning rate, and we choose a value of 0.003 to be in the centre of the stable learning region.

2.3.3 Results

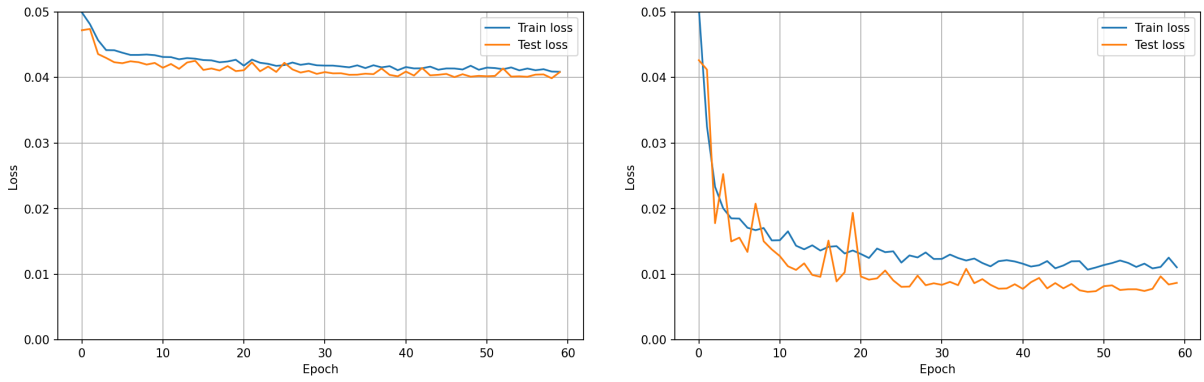
Figure 11 contrasts the training and test loss trajectories of both architectures. The U-Net converges much deeper and faster than the FCN baseline, achieving a final test loss of 0.0086 compared to the FCN's plateaued test loss of 0.0408 after 60 epochs. This is likely because Darcy flow is an elliptic PDE where local pressure is globally dependent on the permeability field $a(\mathbf{x})$ and boundaries. The FCN's limited local receptive field fails to capture this global context.



(a) U-Net architecture

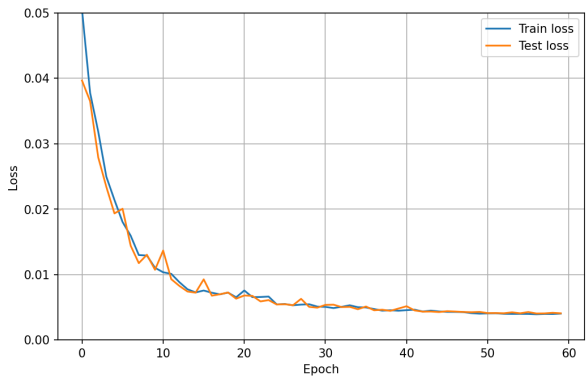
(b) FNO architecture

Figure 10: Learning Rate Finder plots used to systematically identify the optimal starting learning rates for the U-Net (left) and Fourier Neural Operator (right).



(a) FCN: Relative L_2 Loss

(b) U-Net: Relative L_2 Loss



(c) FNO: Relative L_2 Loss

Figure 11: Training and test loss curves for the different network architectures.

Figure 12 visually verifies the U-Net's predictive accuracy. As discussed in subsection 2.2, we plot the ground truth solution $u(\mathbf{x})$ and network approximation $\hat{u}(\mathbf{x})$ in the normalised space to make performance comparison more revealing.

We see that the U-Net architecture provides a very strong approximation of the ground truth, with accurate global and local structure. In contrast the FCN approximation (Figure 9) is much less accurate: $\hat{u}(\mathbf{x})$ has much more jagged edges and struggles to capture the smoothness of the ground truth solution.

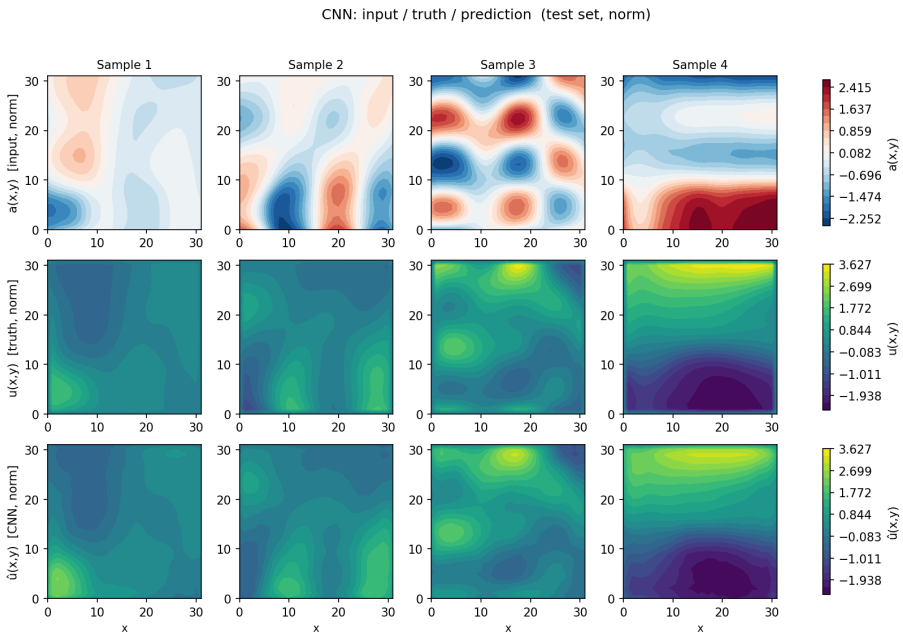


Figure 12: Plot of the input $a(\mathbf{x})$, the normalised FEA ground truth solution $u(\mathbf{x})$ and the normalised approximation derived with a U-Net architecture $\hat{u}(\mathbf{x})$.

2.4 Fourier Neural Operator (FNO)

2.4.1 Architecture choice

The final architecture we implement is a Fourier Neural Operator (FNO), designed to learn the continuous solution operator by operating directly in the frequency domain, with an effectively infinite spatial receptive field. To balance the capacity and ensure a fair structural comparison with our previous CNN networks (which contained roughly 7,300 parameters), we configure the FNO with a channel width of 7 and retain the lowest 4 Fourier modes. The network processes the input through three primary stages:

1. **Grid Concatenation & Lifting:** Standard convolutional architectures are translation-invariant, which poses a challenge for learning PDEs driven by strict boundary conditions. To resolve this, the input permeability field $a(\mathbf{x})$ is concatenated with its spatial coordinates (x, y). A fully connected layer (`nn.Linear`) then 'lifts' this into a higher-dimensional latent space of width 7.

2. **Fourier Integral Operators:** The latent representation is then passed through four consecutive Fourier blocks. To capture both global flow and local variations, each block has two parallel paths:

- *Global Path:* A Fast Fourier Transform converts the spatial data into the frequency domain. The spectrum is truncated to retain only the lowest 4 frequency modes. These modes are multiplied by weights - equivalent to convolution in space - before an inverse FFT returns them to the spatial domain.
- *Residual connection:* A 1×1 convolution (`nn.Conv2d`) and an associated MLP act in a similar way to a parametrised U-Net skip connection. This path preserves fine local details and prevents blurring.

The outputs from both paths are summed and passed through a GELU activation function to provide non-linearity.

3. **Projection Head:** We then use an MLP projection head to compress the 7 channels down to a single output channel, yielding the final predicted pressure field $\hat{u}(\mathbf{x})$.

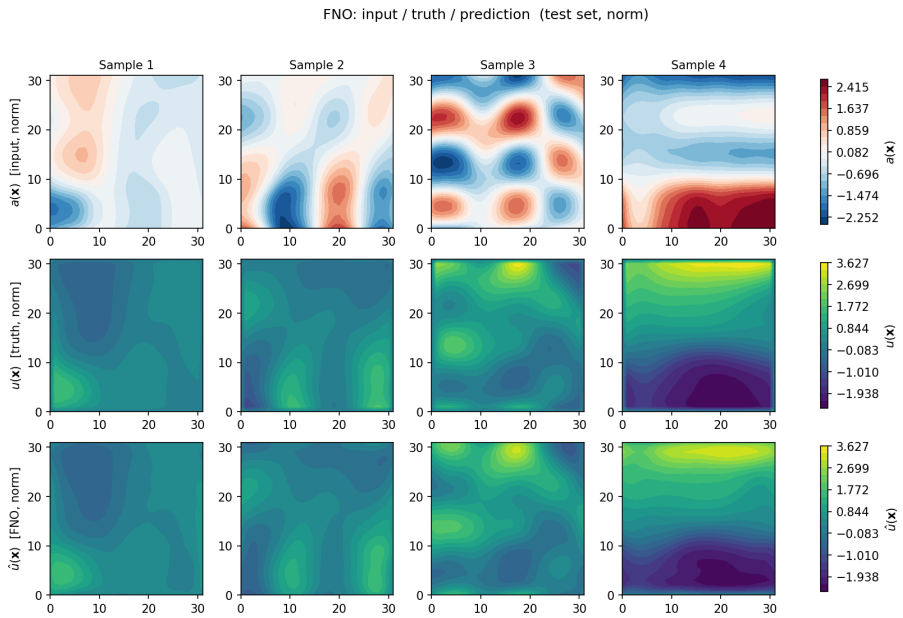


Figure 13: Plot of the input $a(\mathbf{x})$, the normalised FEA ground truth solution $u(\mathbf{x})$ and the normalised approximation derived with an FNO architecture $\hat{u}(\mathbf{x})$.

2.4.2 Optimiser and Learning Rate

We train the FNO using the `Adam` optimiser. Adaptive momentum is particularly crucial for the FNO architecture, as it must simultaneously update parameters in both the continuous frequency domain and the discrete spatial domain. To fix the initial learning rate we use a rate finder as discussed in subsubsection 2.3.2. Based on the region of steepest descent (Figure 10b), we selected a starting learning rate of 0.005.

2.4.3 Results

The training and test loss trajectories for our FNO are presented alongside the convolutional architectures in Figure 11c. Despite their equivalent parameter count ($\approx 7,000$) the FNO significantly outperforms both previous architectures. After 60 epochs, the FNO achieved a highly stable final test loss of 0.0041. This effectively halves the U-Net's test loss (0.0086) and represents a full order-of-magnitude improvement over the FCN baseline

(0.0408). The minimal gap between the FNO train loss (0.0040) and test loss (0.0041) further indicates that the model generalizes exceptionally well without overfitting, validating our heuristic parameter-matching strategy.

This superior predictive capability is visually confirmed in Figure 13. The FNO approximation $\hat{u}(\mathbf{x})$ is virtually indistinguishable from the ground truth $u(\mathbf{x})$, capturing all the local and global features. Ultimately, the FNO's use of convolution via the frequency domain gives it an effectively infinite spatial receptive field. This makes it the most accurate architecture for resolving the global dependencies of the elliptic Darcy flow PDE.